2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00058

Exploring the Performance Improvement of Tensor Processing Engines through Transformation in the Bit-weight Dimension of MACs

Qizhe Wu[1], Huawen Liang[1], Yuchen Gui[1], Zhichen Zeng[1,2], Zerong He[1], Linfeng Tao[1], Xiaotian Wang[1,3], Letian Zhao[1] Zhaoxi Zeng[1], Wei Yuan[1], Wei Wu[1] and Xi Jin[1]*
[1]Department of Physics, University of Science and Technology of China,
[2]University of Washington, [3]Raytron Technology
wqz1998@mail.ustc.edu.cn, xiaotian.wang@raytrontek.com, jinxi@ustc.edu.cn

***Abstract*—General matrix-matrix multiplication (GEMM), serving as a cornerstone of AI computations, has positioned tensor processing engines (TPEs) as increasingly critical components within existing GPUs and domain-specific architectures (DSA). Our analysis identifies that the prevailing architectures primarily focus on dataflow or operand reuse strategies, when considering the combination of matrix multiplication with multiply-accumulator (MAC) itself, it provides greater optimization space for the design of TPEs. This work introduces a novel perspective on matrix multiplication from a hardware standpoint, focusing on the bit-weight dimension of MACs. Through this lens, we propose a finer-grained TPE notation, using matrix triple loops as an example, introducing new methods and ideas for designing and optimizing PE microarchitecture. Based on the new notation and transformations, we propose four optimization techniques that achieve varying degrees of improvement in timing, area, and power consumption. We implement our design in RTL using the SMIC-28nm process. Applying our methods to four classic TPE architectures (include systolic array [20], 3D-Cube [27], multiplier-adder tree [48], and 2D-Matrix [30]), we achieved area efficiency improvements of 1.27×, 1.28×, 1.56×, and 1.44×, and 1.04×, 1.56×, 1.49×, and 1.20× for energy efficiency respectively. When applied to a bit-slice architecture, we achieved a 12.10× improvement in energy efficiency and 2.85× in area efficiency compared to Laconic [38]. Our Verilog HDL code, along with timing, area, and power reports for circuit synthesis in URL: https://github.com/wqzustc/High-Performance-Tensor-Processing-Engines.**

I. Introduction

In the current wave of technological innovation, artificial intelligence (AI) has become central to modern technological development [2]. All these advancements rely on tensor operations, specifically matrix multiplication (MM), which is considered the cornerstone of deep learning and AI. To meet the increasing computational demands of AI, hardware designers are now integrating specialized MM units called tensor processing engines (TPEs) into GPUs [31], CPUs [3], [4], and DSAs [8], [19], [26], [27], [36]. TPE occupies an important part of the area and power in these chips.

The MAC, as a fundamental hardware component, has been extensively studied to better exploit MAC and improve computational performance. The research can be divided into macro-architectural level and micro-arithmetic logic level.

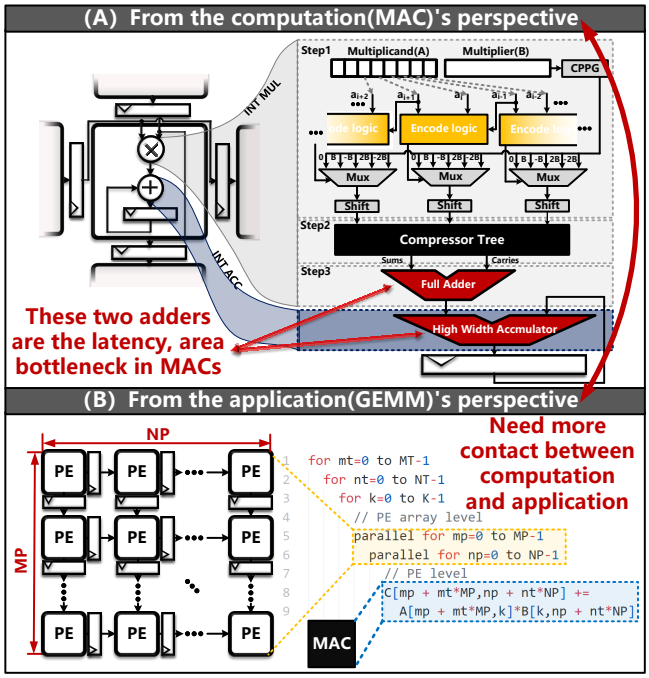

Figure 1: The microarchitecture of INT MAC and MM unit.

On the one hand, architects have designed various architectures for MM based on MAC (Figure 1(B), including 2D-matrix [30], weight stationary (WS) or output stationary (OS) systolic array [13], [20], [21], [33], and 3D-Cube [1], [27]. These architectures have not only gained widespread application in academia but have also been practically deployed in the industry, becoming foundational in TPE design.

On the other hand, arithmetic logic units (ALUs) researchers focus on the approach from the perspective of single computational (Figure 1(A)), striving to develop higher performance multipliers and adders. Typical designs include array multipliers [16], Booth multipliers [23], Baugh multipliers [40], carry lookahead adders [9], carry select adders [6], carry save adders [15], Wallace tree [43] and compressor tree [37], [42].

Architecture design and computational unit design are or-

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00058

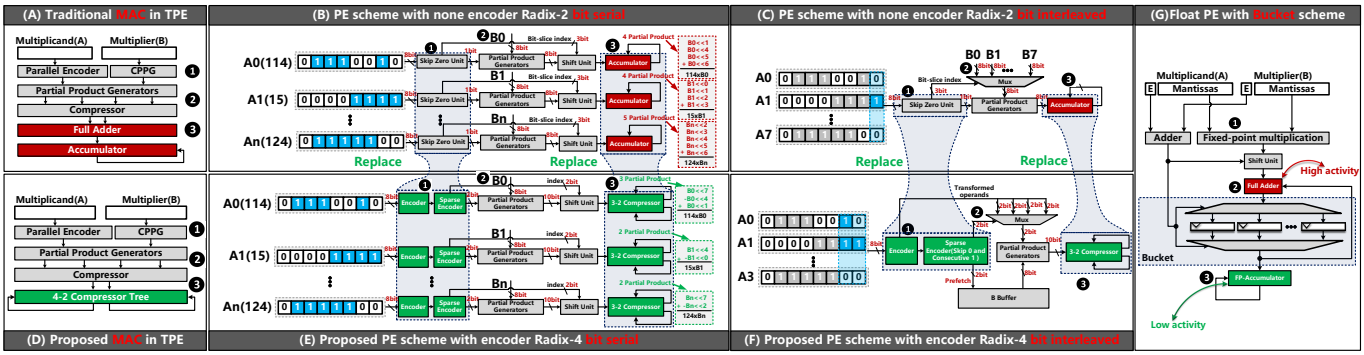

Figure 2: Improvements in microarchitecture compared to other works. (A) Traditional MAC (TPU-Like). (B) and (C) Bit-serial-based computation methods. (D) Optimized MAC. (E) and (F) Optimized bit-serial architectures. (G) Similarities and differences with floating-point optimized schemes. Without showing the DFFs, only Step ❸ includes a pipeline register, while the other steps are single-cycle operations.

thogonal approaches, both of which play a crucial role in enhancing the performance of computer systems. However, current research often focuses solely on one of these two aspects, overlooking the deeper connection between them.

From a comprehensive perspective, traditional MAC (Figure 2(A)) is mainly divided into three stages: ❶ Encode the multiplicand, and generate partial products (PPs). ❷ Compress all PPs to generate the final sum and carry. ❸ Obtain the final result through the processing of full adders and accumulators. It has been proved that the internal maximum logic propagation delay (t_{pd}) and area of high bit-width accumulation units in MACs have become bottlenecks [7] for performance and efficiency (Figure 1(A) step ❸). One solution (Figure 2(G)) proposed by Bucket Getter [28] allows a large number of floating-point additions be converted to fixed-point additions during the reduction (accumulation) phase (Figure 2(G)❷). It significantly reduces the dependency on floating-point accumulators and lowers the activity. This method reduces the power consumption of the floating-point accumulators (Figure 2(G)❸) and further improves energy efficiency within the process element (PE). **QI: However, the issue of high-bit-width fixed-point accumulation bottleneck (t_{pd} and area) remains unresolved in this research.**

Differing from the MAC in parallel, the researchers have proposed bit-slice-based multiplication methods to replace MAC. These methods main include the Radix-2 bit-serial [10], [22], [34], [44], Radix-2 bit-interleaved [5], [29], [39], [46], higher width bit-slice [17] and Radix-4 based slice computation [14], [18], [25], [38]. The Radix-2 bit-serial computation (shown in Figure 2(B)) relies on the sparsity of the multiplicand A and consists of three major steps. Step ❶ is to extract the indices of non-zero bit slices of A and skip zero elements. Step ❷ uses these indices and the multiplier B to generate PPs. Step ❸ is to shift and accumulate these PPs according to their corresponding bit-weight. The computation speed of this method mainly depends on the number of PPs, or the number of non-zero bit slices in A. The Radix-2 bit-interleaved computation method (Figure 2(C)) processes multiple data simultaneously. These data from different operands share the same bit-weight, thus eliminating the need for shift operations. Step ❸ usually consists of an adder-tree or a high bit-width accumulator. The radix-2 multiplication calculation method can maintain high bit sparsity, but it has the disadvantage of requiring a large number of PPs to be accumulated.

Despite the achievements in improving computational efficiency, these bit-slice-based methods still have room for improvement. **QII: Firstly, these methods cannot effectively skip consecutive "1" bit-slice, which may affect computational efficiency in certain cases (under the two's complement representation of a batch of normally distributed data, the number of bit-slices with "1"s in negative numbers is typically greater than those with "0"s). Secondly, the accumulation can still become a performance bottleneck.**

In encoder-based TPE design schemes, such as Pragmatic [5] and Laconic [38], encoder-based strategies are used to generate partial products. Both leverage the bit sparsity of encoded data to accelerate DNNs. However, the fundamental principles underlying the use of encoders for sparsity acceleration have not been thoroughly explored.

To address the aforementioned issues, this study proposes a PE microarchitecture design method tailored for specific tasks. We propose an analytical model of the MAC based on a compute-centric notation. This model assists us in better integrating the concepts presented in Figure 1(A) and (B) from a deeper and more intuitive perspective, as well as mapping the hardware components within the MAC to corresponding primitives, such as parallel encoder, candidate partial product generator (CPPG), partial product generator, shifter, compressor, full adder, and high bit-width accumulator. Through this transformation, we uncover the implicit parallel dimension within the MAC units in the new notation. This dimension was overlooked in the previous TPE design.

By the transformation under the new notation, we design several optimized PE microarchitectures for QI and QII (Figure 2(D)(E) and (F)). In matrix multiplication, MAC usually involves vector reduction in the time dimension. Therefore, before the reduction process is completed, we use compressors

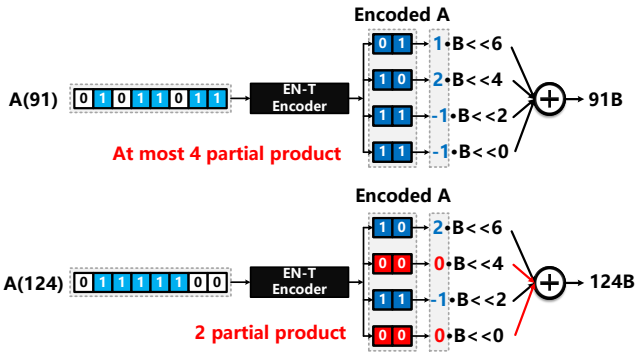

Figure 3: Example of multiplication based on encoding.

Unit	Bit	Area(um^2)	Delay(ns)	TOP(uW)
MAC	20	179.30	1.56	27.1
	24	192.65	1.67	29.2
	28	206.01	1.84	31.4
	32	238.51	1.97	36.3
4-2 Compressor Tree	14	55.92	0.31	8.5
Full Adder	14	51.32	0.34	7.7
Accmulator	20	57.32	0.80	8.6
	24	62.43	0.90	9.4
	28	82.78	0.99	12.3
	32	95.13	1.13	14.3

TABLE I: The main component decomposition in INT8 MAC (tested on SMIC 28nm with a 2ns clock constraint).

[42] to perform accumulation (store the sums and carries in DFFs (Data Flip-Flops)) to replace the full adder in step ❸ of Figure 2(A) and (D). Experiments show that even under high bit-width (≥32bit) accumulation conditions, our design can reduce the t_{pd} of traditional MAC by half in QI. This is because the delay of the half-adder is not dependent on the operand bit-width (shown in Table V).

To solve QII in bit-serial computation, we employ encoding and half-accumulation strategies (Figure 2(E)). In step ❶, encoders are used to replace the original Skip Zero Unit. The encoding can utilize modified Booth encoding (MBE) [23] or EN-T [45] encoding. Subsequently, **sparse encoding is performed on the encoded numbers**, which differs from other bit-slice-based calculations. While other schemes perform sparse encoding on the "0" bit-slice of the multiplicand, we conduct sparse encoding on the encoded number. This is because the encoded number is directly utilized to generate the PPs (Figure 1(A) step ❶).

For comparison purposes, two computational examples are presented in Figure 2(B) and (E), where 114, 15, and 124 are multiplied by three multipliers B_0, B_1 and B_n. Bit-serial computation generates 4, 4, and 5 PPs respectively, requiring corresponding cycles to complete accumulation. In contrast, our proposed method only requires 3, 2, and 2 PPs for these operands. This allows for skipping both zeros and consecutive "1" bit-slices in the multiplicand.

The second improvement involves the use of compressors to replace accumulators in step ❸ of Figure 2(B)(E). This is aimed at optimizing area and timing. In terms of optimization for bit-interleaved methods (Figure 2(F)), this paper employs sparse encoding for the encoded bits of A. **The sparsely encoded indices are used to select the encoded bits, while also utilizing the indices to prefetch B**. Subsequently, PPs are generated by using the encoded bit segments of non-zero multiplicand A and multiplier B. These PPs are then accumulated using 3-2 compressor to optimize computational efficiency.

In summary, our core contributions are as follows:

1) We propose a finer-grained TPE notation and introduce new methods and ideas for designing and optimizing PE microarchitecture in specific applications.
2) Compared to Pragmatic [5] and Laconic [38], we provide a more systematic explanation of the fundamental reasons for bit-sparse acceleration and design a more efficient PE micro-architecture suitable for bit-serial processing, characterized by low area and high frequency. Additionally, we discuss the comparison of other encoding methods for bit-sparse acceleration of the multiplicand.
3) Based on the new notation and transformations, we propose four optimization methods and we implement our design in RTL using the 28nm process. Applying our methods to four classic TPE architectures (include systolic array [20], 3D-Cube [27], multiplier-adder tree [48], and 2D-Matrix [30]), we achieved area efficiency improvements of 1.27×, 1.28×, 1.56×, and 1.44×, and 1.04×, 1.56×, 1.49×, and 1.20× for energy efficiency respectively. When applied to a bit-slice architecture, we achieved a 12.10× improvement in energy efficiency and 2.85× in area efficiency compared to Laconic [38].

II. Background and Motivation

A. High Width Accumulator Represents the Most Challenge

For AI DSA, the performance of the TPE is key to ensuring DNN throughput. For example, in TPU [21], the systolic array and accumulators occupy 36% of the die area and consume 52% of the on-chip power; for Bitwave [39], TPE accounts for 26.8% of the area and 62.5% of the power consumption; for LUTein [18], TPE takes up 27.7% of the area and 51.6% of the on-chip power; for Bucket [28], TPE occupies 59.5% of the area and 33.7% of the on-chip power. Therefore, improving the performance of TPE within the limited on-chip silicon area is crucial for AI DSA.

For TPE, the area and t_{pd} of the MACs are the primary performance bottlenecks. Within the MAC, the compressor tree and the full adder within the multiplier are affected by the multiplication bit-width in terms of logical delay and area, while the bit-width of the accumulator is only related to the number of accumulations required. Consequently, as the bit-width of the accumulator increases, it gradually becomes a limiting factor for the MAC's frequency. According to the circuit synthesis report listed in Table I, it's observed that

NumPPs	4	3	2	1	0
MBE [12]	81 (31.6%)	108 (42.2%)	54 (21.1%)	12 (4.7%)	1 (0.4%)
EN-T [45]	72 (28.1%)	108 (42.2%)	60 (23.4%)	15 (5.9%)	1 (0.4%)
NumPPs	**{8,7}**	**{6,5}**	**4**	**{3,2}**	**{1,0}**
bit-serial	9 (3.5%)	84 (32.8%)	70 (27.3%)	84 (32.8%)	9 (3.5%)

TABLE II: The number of partial products (NumPPs) under different encoding within the range of INT8 ($-128 \sim 127$).

Distribution	$\mathcal{N}(0, 0.5)$	$\mathcal{N}(0, 1.0)$	$\mathcal{N}(0, 2.5)$	$\mathcal{N}(0, 5.0)$
EN-T	2.27	2.22	2.26	2.23
MBE	2.46	2.41	2.45	2.42
bit-serial(M)❶	3.52	3.52	3.52	3.53
bit-serial(C)❷	3.99	3.98	3.98	3.98

❶ Operand with complement representation.
❷ Operand with sign-magnitude representation.

TABLE III: The average NumPPs of each multiplicand in different encoding based on the normal distribution matrix.

with the bit-width of the accumulator increases in MAC, the predominant factors constraining the performance progressively transition to the area and delay of the accumulator. For instance, in a 32-bit accumulation, the logical area occupied by the full adders and accumulator accounts for 61.4%, and the logic delay is as high as 74.6%, severely restricting the frequency of the MACs.

B. Fine-grained Description of the TPE Microarchitecture

The RTL-based description of the TPE microarchitecture is overly detailed for designers to understand the acceleration mechanism at the algorithm level, while the hardware block diagram representation is too abstract for the underlying implementation level. Using a notation between the hardware block diagram and RTL can help designers understand the acceleration mechanism at both levels. However, existing design space representations [24], [32], [35], [47], [50] focus on architecture with MAC as the basic unit and don't explicitly represent the reduction logic (adder-trees) brought by spatial unrolling and the reduction logic in PEs constitutes a significant portion of the PE area and serves as a critical factor that affects timing. These limitations make it difficult to capture acceleration opportunities at the PE microarchitecture level. Therefore, there is a need to develop a more comprehensive notation for TPE to capture data flow and operational specifics in detail, enabling further exploration of hardware architecture optimization methods under specific application conditions.

C. Sparsity Acceleration Based on the Encoding Principle

In previous research based on bit-serial methods [17], [29], [39], the sparsity of bit-slice was often used to discuss potential speed improvements while overlooking the number of partial products (NumPPs) in multiplication. However, the NumPPs directly influence hardware delay and area for parallel multiplication, as well as the number of cycles needed for serial multiplication.

For a Radix-4 parallel multiplier, an n bit multiplicand processed by an encoder (MBE [12] or EN-T [45]) will produce $\frac{n}{2}$ PPs. Taking Radix-4 EN-T as an example (Figure 3(A)), for INT8 multiplication, the multiplicand A (in two's complement) generates four 2-bit encoded numbers after passing through the encoder. For instance, with 91, the encoded numbers are {1, 2, -1, -1}, corresponding to PPs coefficients with bit-weight $\{2^6, 2^4, 2^2, 2^0\}$. Therefore, multiplying the multiplier B by 91 can be expressed as four PPs: 91B = (B<<6) + (2B<<4) + (-B<<2) + (-B). The candidate PPs only need to compute {-2B, -B, 0, B, 2B} in MBE for selection by the encoded numbers, and the shifter is responsible for shifting the generated PPs by the corresponding bits weight.

However, not all numbers will produce 4 non-zero PPs (Figure 3(B) 124 can be encoded as {2, 0, -1, 0}, so 124B = (2B<<6) + (-B<<2)). We counted the NumPPs generated by the two Radix-4 encoders and Radix-2 bit-serial within the INT8 range (Table II). Under MBE, 175 ((108 + 54 + 12 + 1)/256 $\approx$ **68.4%**) numbers generate 3 or fewer non-zero PPs during multiplication. Under EN-T, 184 ((108 + 60 + 15 + 1)/256 $\approx$ **71.9%**) numbers generate 3 or fewer non-zero PPs. Similarly, Radix-2 bit-serial complement multiplication can be viewed as generating PPs without encoding. Only 93 ((84 + 9)/256 $\approx$ **36.3%**) numbers generate 3 or fewer non-zero PPs during multiplication.

To assess the overall operational cost of batch data, we use the average NumPPs as a metric. Fewer PPs lead to faster computation and lower power consumption. In Table III (matrices size 1024×1024), the average NumPPs for EN-T and MBE range from 2.22 to 2.45. Therefore, for large-scale matrix multiplication, we break down the vector dot product operation into two key steps: ❶ Generating non-zero partial products. ❷ Reduction of partial products. For parallel MAC, the multiplicand is encoded during the computation of the vector dot product, and the partial products are expanded spatially as an implicit dimension for parallel processing and reduction. This process ignores the scenario where some of the generated partial products are zero. Thus, by decomposing the multiplication operation into sequential partial product reduction combined with a non-zero partial product generator, the number of operations in matrix multiplication can be significantly reduced.

III. Proposed Methodology

In this study, we employ a compute-centric notation that closely resembles software pseudocode to improve comprehensibility. This notation is utilized to depict the microarchitecture of TPEs by incorporating the bit-weight dimension (referred to as BW) of MACs. Subsequently, we will examine the new hardware primitives introduced by the BW and provide an example of TPEs utilizing a 2D-OS dataflow within the notation. Our goal is to offer a clear and professional per-

PRIMITIVE	DESCRIPTION
$half\_reduce(I_1, I_2, \ldots, I_n)$	Compressor tree, with n inputs (I_1 to I_n) and 2 outputs (sum and carry).
$add(I_1, I_2)$	Full adder, with 2 input and 1 output.
$accumulate(I)$	Accumulator, unlike the full adder, has inputs that depend on the output of the previous cycle.
$encode(I, i)$	Encoder, outputs candidate PPs selection signal. I represents bit-slice of multiplicand, and i is the i-th bit weight. In MBE, $i \in [0, 3]$ and I consists of $[2i-1 : 2i+1]$ slice from multiplicand.
$map(I, sel)$	CPPG and Mux, "map" generates the PPs based on the input I through selects the corresponding PPs based on the sel.
$shift(I, i)$	Shifter, I denotes the data to be shifted, while i is the configuration. I will be shifted to the left by $2i$ bits in MBE.

TABLE IV: Components described by hardware primitives.

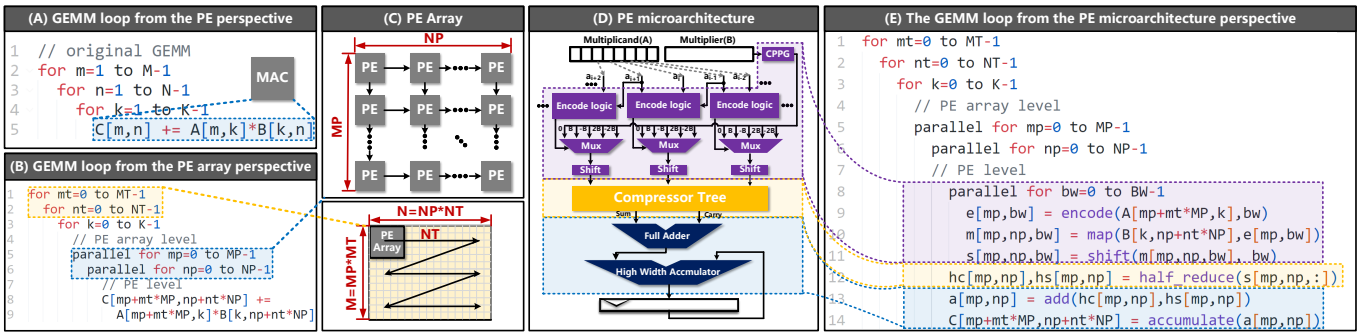

Figure 4: The GEMM loop from the PE microarchitecture perspective.

spective on how the BW of MACs impacts the performance of TPEs.

A. BW Dimension and New Hardware Primitives

In a multiplier, the calculation process can be visualized as a multiplicand expanded into multiple sub-operands, which are then multiplied in parallel with another operand (resulting in the PPs of different bit-weights), and finally reducing all PPs to obtain the result. This can be expressed as follows:

$$C = A \times B = \sum_{bw=0}^{BW-1} SubA_{bw} \times B. \quad (1)$$

It should be noted that the size of BW and the form of $SubA_{bw}$ are related to specific encoding methods. In this paper, we focus on the acceleration opportunities brought by the BW dimension, rather than the design of specific encoding methods. Here, we only use two examples to illustrate that Eq. (1) can broadly represent the multipliers. For an 8-bit MBE [49], $SubA_{bw}$ and BW are as follows:

$$SubA_{bw} = (-2a_{2bw+1} + a_{2bw} + a_{2bw-1})2^{2bw}, BW = 4, \quad (2)$$

where a_{2bw} represents the $2bw$-th bit of A. For an 8-bit complement bit-serial method [17], $SubA_{bw}$ as follows:

$$SubA_{bw} = \begin{cases} a_{bw}2^{bw}, & if\ bw \neq BW-1 \\ -a_{bw}2^{bw}, & if\ bw = BW-1 \end{cases}, BW = 8. \quad (3)$$

Based on Eq. (1), the multiplier exposes its implicit dimension BW, which represents the number of sub-operands of A. Based on Eq. (2) and (3), each sub-operand can be represented as the encoding of a bit-slice multiplied by a weight. Therefore, we call this hidden dimension the bit-weight dimension. In matrix multiplication, we apply Eq. (1) to obtain the following form:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k}B_{k,n} = \sum_{k=0}^{K-1}\sum_{bw=0}^{BW-1} SubA_{m,k,bw}B_{k,n}. \quad (4)$$

The primary objective of this paper is to utilize the microarchitectural hardware diversity uncovered by BW in order to investigate potential optimization opportunities for TPEs. To precisely demonstrate the impact of the new dimensional transformation on hardware design, we introduced new primitives and explicitly represented these components in the notation. The primitives are shown in Table IV.

In the process of multiplication, there are four key components involved in generating the PPs: encoders, CPPGs, multiplexers, and shifters. We utilize the terms **"encode"**, **"map"** and **"shift"** to denote these components. The reduction logic in the MAC, including the compressor tree, full adder, and accumulator, not only takes up a significant area within the PE but also has a crucial impact on timing. In light of this, we have introduced **"half_reduce"**, **"add"**, and **"accumulate"** to explicitly represent the reduction logic in the notation.

In the following sections, we will investigate how BW transformation impacts TPE microarchitecture by analyzing these components and their relevance. Based on this analysis, we will develop more efficient parallel hardware.

B. Matrix Multiplication from a Microarchitecture Perspective

Starting with the traditional triple-nested loop of MM (Figure 4(A)) and the compute-centric notation form (Figure 4(B)),

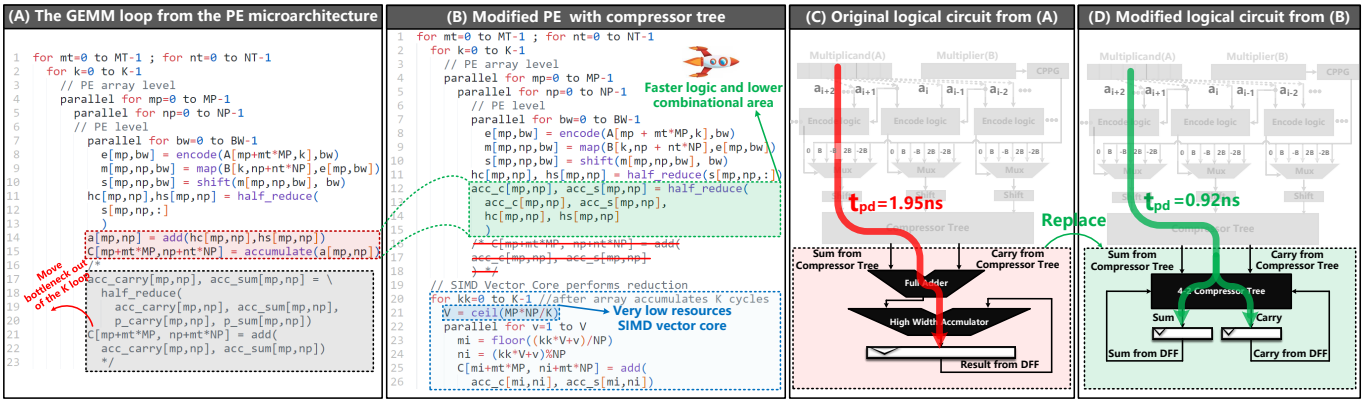

Figure 5: The proposed optimization architecture 1 (OPT1).

we propose Figure 4(E) as the new notation for the TPE by introducing the BW and new computational primitives.

As illustrated in Figure 4(B), the dimensions M and N are split into 4 sub-dimensions. The M_T and N_T are the temporal sub-dimensions of M and N, and the suffix or subscript "T" refers to temporal dimension. The data is iterated in the zigzag form, and $N_P \times M_P$ loop instances are processed and iterated once at each step within the PE array. The M_P and N_P are the spatial sub-dimensions, while the suffix or subscript "P" refers to spatial unrolling dimensions. The "parallel" in pseudo-code means this dimension is mapped to PE array.

Combined with the PE microarchitecture in Figure 4(D), the MAC micro-operation (Figure 4(E)) using primitives from Table IV. The **"encode"** generates the select signal for the Mux, while the **"map"** produces candidate PPs for the Mux inputs and selects the final PPs. The following equation was derived from these basic primitives:

$$\begin{aligned} C_{m,n} &= \sum_{k=0}^{K-1} \sum_{bw=0}^{BW-1} map(B_{k,n}, encode(A_{m,k,bw}))shift(bw) \\ &= \sum_{bw=0}^{BW-1} shift(bw) \sum_{k=0}^{K-1} map(B_{k,n}, encode(A_{m,k,bw})). \end{aligned} \tag{5}$$

It is obvious that the **"shift"** is independent to N and relevant to K, M and BW in Eq. (5), so that **"shift"** can be decoupled from **"encode"** and **"map"**, and be outer level of the K dimension. The movement of **"shift"** helps to reduce the number of the **"shift"** in the array. This inspires us to change our position in notation to explore new architectural designs.

In Figure 4(D), the multiplexer outputs one of the candidate PPs based on the select signals. If we represent the select signals as a one-hot vector, then the selection can be viewed as a dot product of the candidate PPs and select vector. Eq. (5) can be decomposed as follows:

$$C_{m,n} = \sum_{bw=0}^{BW-1} shift(i) \sum_{k=0}^{K-1} \overrightarrow{enc_{m,k,bw}} \lozenge \overrightarrow{prod_{k,n}}, \tag{6}$$

where the symbol $\lozenge$ refers to the selection operation. It is a non-commutative operation, as the inputs and select signal of the multiplexer cannot be reversed. The **"encode"** is independent of N and can be placed outer of the N dimension. The **"map"** contains the selection operation for the multiplexer instance, so it can only be located in the innermost loop.

Other notations derived from Einsum notation focus more on data reuse above the MAC level. The notation we proposed can represent the hardware implementation of MAC in a fine-grained manner, which is able to represent bit-slice accelerators and allows for the representation of intermediate signal reuse within MACs. Based on the preceding analysis of the legality of component positions and nested levels, we can change the position and order of components. Intuitively, changing the nested levels of components can change the number of components, while changing the order can change the critical path of MAC, thereby bringing a new design space dimension to TPE. Just like the skip-zero in a bit-serial multiplier, under our notation, we can convert the BW dimension to the temporal dimension to skip zero partial products and utilize the sparsity discussed in Sec. II-C.

In the next section, we will optimize the PE microarchitecture using these primitives step by step and demonstrate the entire optimization process.

IV. Proposed Architecture

This section delves into optimizations based on the new notation to uncover the potential for latency or area improvements. Conventional design space exploration methods mainly focus on loop transformations and changing spatial mapping dimensions. In contrast to previous studies, our study provides a more detailed analysis of MACs and proposes four orthogonal optimization techniques aimed at enhancing TPE performance within the current notation framework.

A. Half Compress Accumulation Reduction (OPT1)

In traditional MAC-based TPE (Figure 5(A)), the **"accumulate"** follows the **"add"** because the compiler needs to keep the multiplier atomic. Accumulating the output of a full adder is common but costly due to high bit-width results. Fortunately, from a MAC's perspective, it is possible to reverse the order of

Component	Width	Area(um^2)	Delay(ns)
4-2 Compressor Tree	14	52.92	0.31
	16	60.98	0.32
	20	77.11	0.32
	24	93.99	0.32
	28	110.12	0.32
	32	126.25	0.32

TABLE V: Timing and area of the compressor on SMIC 28nm.

reduction (**"accumulate"**) and **"add"**. This means replacing codes in the red box (line 14 $\sim$ line 15) with those within the gray box (line 16 $\sim$ line 23) in Figure 5(A) keeps the result correct. The reorder results in a faster and smaller logic circuit for the reduction: the accumulation in the compressor tree.

It is essential to note that the **"add"** depends solely on the accumulated acc_c and acc_s (Figure 5(A) line 17). Therefore, the result of the **"add"** is not needed until the final iteration of the K dimension, when the accumulation of acc_c and acc_s is not completed, the computation of the **"add"** is redundant.

Inspired by the above, we propose an optimization strategy illustrated in Figure 5(B). This strategy uses the half-add operation during the reduction process of K dimension, **which ensures that the logic delay is independent of the cumulative bit-width**, reducing the need for full adders and accumulators. With only one valid output generated within K cycles per PE, fewer **"add"** operations are needed to merge the acc_s and acc_c at the same level of K dimension. The external full adders (typically a SIMD vector core) outside the PE array handle these **"add"** operations and work with TPE in parallel. Since the SIMD vector core only accesses the data for every K cycle, hence, fewer hardware resources ($\lceil (M_P N_P / K) \rceil$) are required to accomplish these tasks.

The hardware architecture of the original PE is illustrated in Figure 5(C), while the proposed in Figure 5(D). This enhancement involves replacing the full adder and accumulator, which currently account for a significant portion of the critical path delay (t_{pd}) and area within the PE, with a single 4-2 compressor tree. With a clock constraint of 2ns, we are able to reduce the t_{pd} from 1.95ns to 0.92ns (for INT8 multiplication and INT32 accumulation synthesis on SMIC-28nm), and easily achieve a clock frequency exceeding 1GHz. This is because the delay of the compressor, composed of half adders (without carry chains), is independent of bit-width (shown in Table V).

B. Reduction under the Same Bit-weight (OPT2)

According to Eq. (5), the **"shift"** is correlated with the BW. When rearranging loop unrolling, it's important to keep the **"shift"** within the BW loop. This means restructuring the BW as an outer loop that extends beyond the PE array while adjusting the **"shift"** in an outer loop. This positional transformation reduces the number of shifters and decreases the bit-width of subsequent components (compressor tree and DFFs) in PE, leading to a smaller area.

Please note that the BW was initially adorned with "parallel", indicating spatial unrolling in hardware. Simply reordering the BW dimension to the outer level of the K dimension

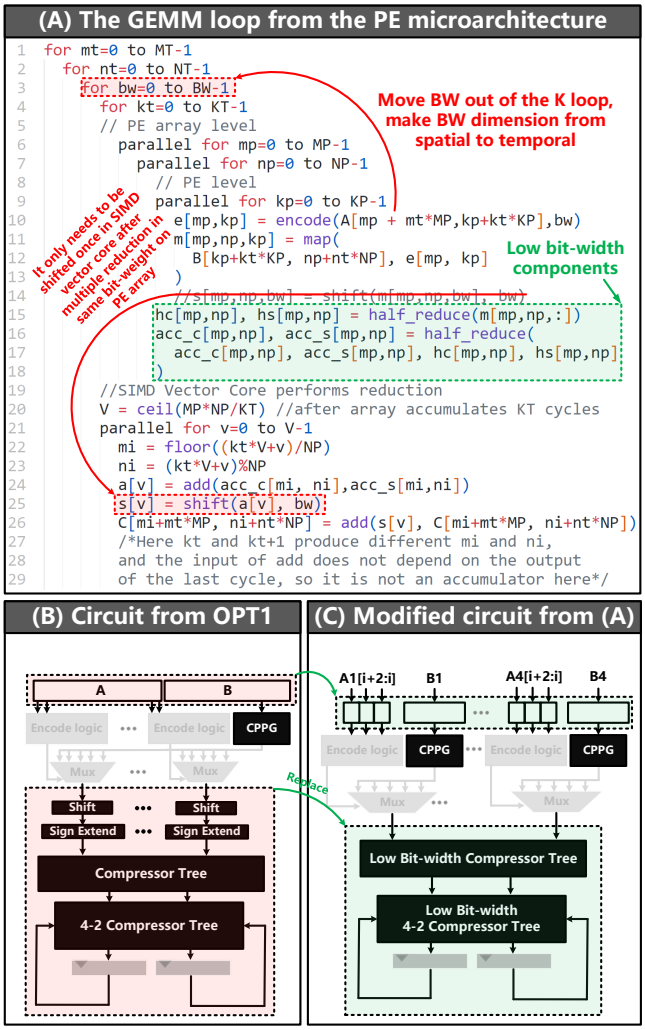

Figure 6: The proposed optimization architecture 2 (OPT2).

will generate error reduction logic, as the **"half_reduce"** is the reduction logic of BW and needs to be at the same level as BW. When moving BW to the outer level, its dimension should be transformed into a temporal dimension.

To maintain the throughput of the PE array, we partition the dimension K into K_P and K_T (Figure 6(A) line 9 and 4), utilizing K_P to fill the gaps in BW. Therefore, the **"half_reduce"** in Figure 6(A) line 15 and 16 represents the reduction logic for dimensions K_P and K_T, respectively. Similar to relocating the **"add"** in OPT1 (Sec. IV-A), we can also transfer the **"shift"** to the SIMD vector core, requiring only a single shift after dimension K_T has finished reduction.

After the **"shift"**, full adders are required to reduce the shifted PPs in order to ensure the correctness of the computation (Figure 6(A), line 26). The reason for using an **"add"** primitive here instead of **"accumulate"** is that the data stream from the PE array ensures that the indexes accessed by the SIMD vector core are unique from each other, and there are no accumulation dependencies within K cycles in SIMD core.

Therefore, pipelined full adders can be implemented here

PRIMITIVE	DESCRIPTION
$sparse(I_1, I_2, \ldots, I_n)$	Outputs the indexes of non-zero inputs, e.g. $[1,3] = sparse([0,1,0,2])$.
$sync()$	Synchronizing sparse computation of the PE subarrays.

TABLE VI: Sparse and synchronization primitives.

to boost frequency, while the pipeline design in Figure 5(A) is meaningless due to data dependencies.

With the shifter eliminated in the PE, the bit-width of input and output of **"half_reduce"** in Figure 6(A) lines 15 $\sim$ 18 are also reduced, which further decreases the logic area (hardware architecture is shown in Figure 6(B)(C)).

However, there are two obvious drawbacks to this improvement. The first drawback is the increased bandwidth of PE. The second drawback is the potential increase in the number of CPPGs and input DFFs for operand B, which would occupy the additional area, for array designs, these additional areas can be shared among multiple PEs. And these drawbacks will be addressed step by step in the following subsections. In general, mapping the BW to a temporal loop and reordering it to the outer loop of the K dimension can reduce the area of shifters, compressor trees, and output DFFs. When considering the sparsity of encoding, temporal unrolling of the BW could prove to be greatly advantageous.

C. Acceleration with the Sparsity of Encoding (OPT3)

In Sec. II-C, we discussed the impact of zero bit-slices on operand encoding and their effect on average NumPPs in multiplication. Our proposed notation allows us to explore how encoding sparsity improves performance. Additionally, to address the drawback in OPT2, we introduce OPT3 in this section as a basis and fully resolve the issue with OPT4 in the next section. To describe the modified architecture, we introduce the **"sparse"** and **"sync"** primitives (in Table VI).

The term **"sparse"** is used to compress inputs and obtain the indices of non-zero inputs. In contrast to previous work [17], [18], [29], [39], we use **"sparse"** for encoding numbers, while other works use it for multiplicands.

To accumulate the non-zero PPs in reduction of K, we store the encoded number in the input DFFs of the PE (Figure 7(C) step ❶). Then, an additional sparse encoder is introduced to output the non-zero index of the encoding number, as shown in Figure 7(C)(D) step ❷. This index is then used as a selection signal for the non-zero PPs and multipliers B in step ❸. Finally, a compressor is used to complete the accumulation. According to Table III, it takes only an average of 2.2 clock cycles to complete this equivalent multiplication, making the TPE more lightweight and able to run at higher frequencies.

Since PEs in the same column can share the same multiplicand A in Figure 7(B), their computation time is uniform within a column but may differ across columns. Therefore we introduce the **"sync"** to synchronize across PE columns. The **"sync"** blocks PE columns that finish earlier until all columns in the array are completed, indicating that

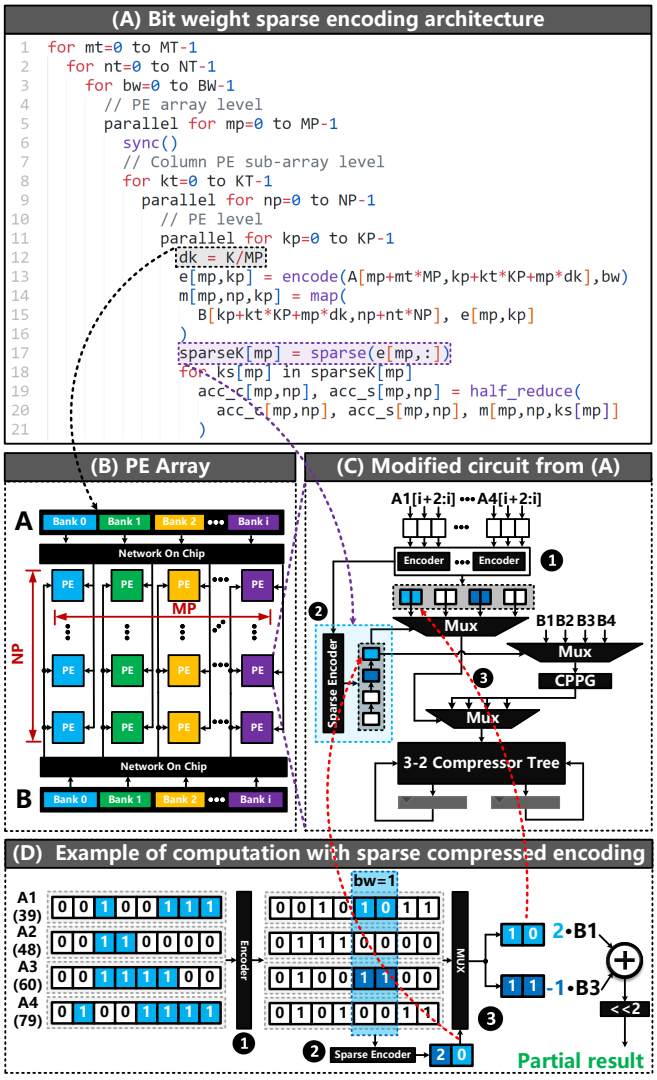

Figure 7: The proposed optimization architecture 3 (OPT3).

the PE array is synchronized once every T_{sync} cycle at most. T_{sync} is determined by multiplying temporal loop cycles (K_T) and the number of unrolling operands in each PE (K_P), as unrolling operands are serialized to eliminate redundant multiplications by zero. For example, in Figure 7(A), column PEs are synchronized once at most every $K_T \times K_P$ cycles.

In Figure 7(A), we place the **"sync"** at the same level as the K_T and below the spatial dimension M_P. This means that PEs in the same column share operand A. After K_T iterations, the PE array synchronizes once. However, different columns work asynchronously, which can lead to bank conflicts. To avoid this, we switch the layout of A from MK to $K_1 M_T K_2 M_P$, where K_1 and K_2 are sub-dimensions of the K with sizes of M_P and K/M_P. Similarly, the layout of B (KN) is mapped to $K_1 N_T K_2 N_P$. The elements of A with the same index in K_1 are stored in the same bank, and the index difference between two adjacent banks will be dk in the K dimension. So is the operand B. With this layout strategy, each column can access N_P elements in A and N_P elements in B in the same bank

without data conflicts (Figure 7(A) line 12 $\sim$ 16). Here, we omit the primitive representation of the SIMD core.

Although computation time may vary for different PE columns, the total time will converge as long as there are sufficient elements along the K dimension. To analyze the expected time between synchronizations, we define the number of non-zero PPs as a random variable X follows $X \sim B(K, 1-s)$ where the sparsity of encoding is s. Therefore, we can obtain the $\mu = K(1-s)$ and $\sigma = \sqrt{Ks(1-s)}$.

For the M_P columns, let T_i be the computation time for the i-th column when executing K_T inputs. The interval between two **"sync"** operations is $T_{sync} = max(T_1, T_2, ..., T_{M_P})$. The T_i is identically and independently distributed, and the cumulative distribution function $F(t) = P(T_{sync} \leq t)$ can be obtained as follows:

$$F(t) = \prod_{i=0}^{M_P} P(T_i \leq t) = \prod_{i=1}^{M_P} \sum_{j=0}^{t} \binom{K}{t} s^{K-j}(1-s)^j. \quad (7)$$

Therefore, the mathematic expectation of T_{sync} is:

$$E[T_{sync}] = \sum_{t=0}^{K} tP(T_{sync} = t) = K - \sum_{t=1}^{K-1} F(t). \quad (8)$$

Based on Eq. (8), when synchronization is performed at the granularity of the PE columns, acceleration based on encoding sparsity can result in an average reduction of $\sum_{t=1}^{K-1} F(t)$ cycles. In practical applications, such as DNN, the weights or inputs typically follow a normal distribution around zero. Taking a middle layer of ResNet-18 as an example and converting it into an MM through img2col, the reduction dimension size of the weights is 576 ($192 \times 3 \times 3$). The sparsity of the weights is 0.38 when using EN-T [45] for encoding. According to Eq. (7) and Eq. (8), the expected T_{sync} would be 381, which represents a time saving of approximately 33.84%.

In summary, directly analyzing the encoding number is more effective than multiplicand, as it is used to directly generate PPs, and skip consecutive "1" bit-slices not only zero (e.g."01111100→10001100" in Figure 3). In OPT2, the PE computes PPs in parallel, requiring a compressor tree in the K_P. In contrast, OPT3 performs serial computations on the sparse compressed K_P dimension. This change eliminates the need for a K_P-input compressor tree and transforms the 4-input compressor tree into a 3-input one in Figure 7(C). However, while this reduces the logic area, it does not address the increased bandwidth of operand B. Additional optimizations are proposed in OPT4 below to efficiently address this issue.

D. Extracted and Shared Encoder (OPT4C and OPT4E)

Based on the analysis in Sec. III-B, we can rearrange the order of the N_P and the K_P (Figure 8(A) line 9 $\sim$ line 15), and move the **"encode"** and **"sparse"** to the outer level of N_P dimension (here, we omit the primitive representation of the SIMD core.). Only the **"map"** and the **"half_reduce"** remain in the innermost loop. Essentially, as operand A is broadcast across the PE columns, PEs in each column can share the same encoder and sparse encoder (Figure 8(B)). By placing the shared encoder outside the PE array, the duplication area of the encoder is reduced in each PE, which also reduces the bandwidth requirement of operand A. Additionally, with the sparse encoder located outside the PE array, the memory can recognize the sparsity of encoded operand A, and prefetch operand B by non-zero indices. With the out-of-plane encoder, the increased input in OPT2 is split and fed to PEs in a sequential manner. Each PE has access to only one shared encoding A and its corresponding prefetched B.

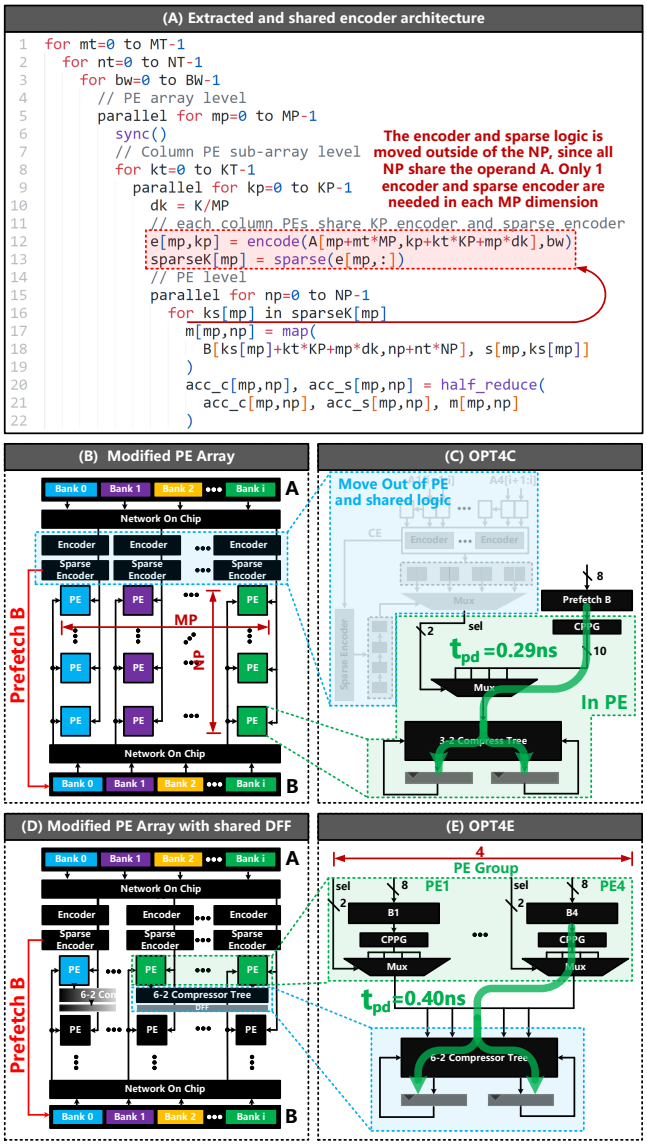

Figure 8: The proposed optimization architecture 4 (OPT4).

At this stage, PE contains only a CPPG, a Mux and a 3-2 compressor tree (Figure 8(C)), with a delay of only 0.29ns. The input ports of each PE include a 2-bit selection signal "sel" and an 8-bit operand B, which reduces the bandwidth requirement. Moreover, compared to OPT3, it eliminates the encoding power consumption within each PE.

We propose an improved version with a higher computing

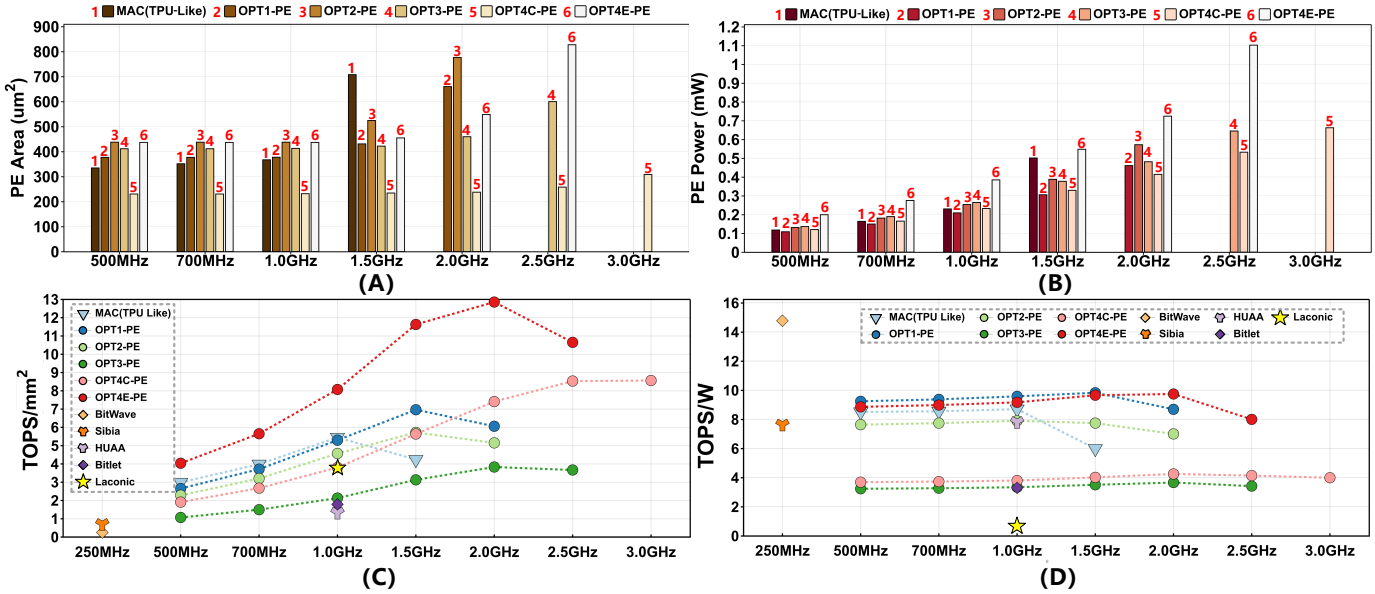

Figure 9: (A) PE area. (B) PE power consumption. (C) PE area efficiency curve. Under different clock constraints compared with the state-of-the-art. (D) PE energy efficiency curve. Under different clock constraints compared with the state-of-the-art.

density as shown in Figure 8(D)(E). We arrange 4 PEs in the same row into a PE group (PE_g), and the PE_g shares one compressor tree and same DFFs. Four 3-2 compressor trees in PE_g are merged into one shared 6-2 compressor tree. At this point, the external Encoder and Sparse Encoder of the PE Array, together with the CPPG in PE_g, form a non-zero partial product generator. This enables significant multiplication operation efficiency in large-scale MM, and the shared encoder also simplifies the internal logic of each PE, resulting in extremely low latency (easily up to 2GHz). Although there is a slight increase in the logical delay from 0.29ns to 0.40ns compared to Figure 8(C), this reduces the DFFs area and corresponding flip-flop power consumption in the PE Array by three-quarters, thereby improving the overall computational density and energy efficiency.

V. Experiment

A. Experimental Setup

1) Hardware modeling: We implement our design in RTL and then synthesize it using the Synopsys Design Compiler with the SMIC 28nm-HKCP-RVT technology at an operating voltage of 0.72V. We utilize Cadence Innovus for placing and routing. Next, we use VCS to generate an FSDB waveform based on the given stimulus signals and use the waveform along with the optimized netlist, GEF and GDS files, the corresponding process corner, and physical libraries to evaluate hardware power consumption and timing using the PrimeTime PX tool. Finally, we employ Calibre to perform layout DRC/LVS checks. OPT4E chip layout is shown in Figure 10.

2) Experimental Arrangement: In the second subsection, we evaluate the frequency characteristics of a single PE under given timing constraints, with a specific timing margin (8% $\sim$ 10% relative to the clock period). This evaluation includes

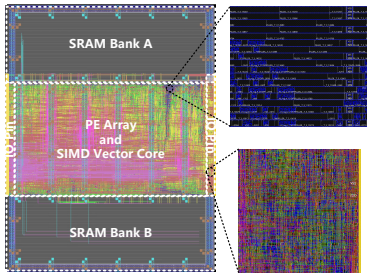

Figure 10: OPT4E chip layout (include IO and fillers).

five microarchitectures (OPT1, OPT2, OPT3, OPT4C, OPT4E) under INT8 MUL and INT32 ACC, which are compared with other PE microarchitectures (MAC (TPU-Like [20]), Laconic [38], Bitlet [29], Sibia [17], Bitwave [39], HUAA [11]) under INT8. The benchmarks include area, power, area efficiency, and energy efficiency. The test data consists of a normally distributed dense vector. The performance metric is the number of element-wise multiply-accumulate operations per second, and power is measured as the average power during the test. The area and power measurements include PE input/output DFFs, combinational logic, and clock networks.

In the third subsection, we test the dense matrix multiplication performance of the PE Array, using the same data distribution and performance-power testing methods as for the PE. Since the OPT1 and OPT2 are optimized for traditional PE arrays, the evaluations of the OPT1 and OPT2 are performed in four classic microarchitectures (systolic array (TPU [20]), 3D-Cube (Ascend [27]), multiplier-adder tree (Trapezoid [48]), and 2D-Matrix (FlexFlow [30])). The Cube contains 1000 ($10\times10\times10$) PEs, and others are 32×32 PEs. We also evaluate the performance of OPT3, OPT4C, and OPT4E ($32\times32 PE_g s$) in comparison with other bit-slice architectures.

Others	TPU$^{\diamond}$	Ascend$^{\diamond}$	Trapezoid$^{\diamond}$	FlexFlow$^{\diamond}$	Laconic$^{\blacklozenge}$	Bitlet$^{\diamond}$	Sibia$^{\blacklozenge}$	Bitwave$^{\diamond}$
Frequency(MHz)	1000	1000	1000	1000	1000	1000	250	250
Area(um^2)	370631	320783	283704	332848	213248	415800	1069000	861681
Power(W)	0.25	0.24	0.22	0.28	1.21	0.23	0.10	0.01
Peak Performance(TOPS)	2.05	2.05	2.05	2.05	0.81	0.74	0.77	0.22
Energy Efficiency(TOPS/W)	8.05(×1.00)	8.21(×1.00)	9.31×1.00)	7.29(×1.00)	0.67(×1.00)	3.29(×4.91)	7.65(×11.42)	14.77(×22.04)
Area Efficiency(TOPS/mm^2)	5.53(×1.00)	7.22(×1.00)	7.22(×1.00)	6.15(×1.00)	3.77(×1.00)	1.79(×0.47)	0.72(×0.19)	0.25(×0.07)
Ours	**OPT1$^{\diamond}$ (TPU)**	**OPT1$^{\diamond}$ (Ascend)**	**OPT1$^{\diamond}$ (Trapezoid)**	**OPT1$^{\diamond}$ (FlexFlow)**	**OPT2$^{\diamond}$ (FlexFlow)**	**OPT3$^{\diamond}$**	**OPT4C$^{\diamond}$**	**OPT4E$^{\blacklozenge}$**
Frequency(MHz)	1500	1500	1500	1500	1500	2000	2500	2000
Area(um^2)	436646	332185	271989	373898	347216	460349	259298	672419
Power(W)	0.37	0.24	0.22	0.38	0.35	0.70	0.51	0.89
Peak Performance(TOPS)	3.07	3.07	3.07	3.07	3.07	1.80	2.25	7.22
Energy Efficiency(TOPS/W)	8.41(×1.04)	12.82(×1.56)	13.89(×1.49)	8.08(×1.11)	8.77(×1.20)	2.57(×3.83)	4.41(×6.58)	8.11(×12.10)
Area Efficiency(TOPS/mm^2)	7.04(×1.27)	9.25(×1.28)	11.29(×1.56)	8.22(×1.34)	8.85(×1.44)	3.91(×1.04)	8.68(×2.30)	10.73(×2.85)

$^{\diamond}$ Reports on timing, power, and area after logic synthesis.
$^{\blacklozenge}$ Reports on timing, power, and area after placing and routing by chip layout.

TABLE VII: Comparision with state-of-the-art in PE Array level on matrix multiplication.

B. PE Comparision

1) Area and area efficiency:

2) Comparison Method: For computation arrays in TPU, Ascend, Trapezoid, and FlexFlow, we utilize Verilog HDL to recurrent their design. In the case of bit-slice architectures such as Laconic, Bitlet, Sibia, and Bitwave, we extract the area and power breakdowns of the PE arrays from their respective papers. When dealing with process nodes other than 28nm, the results are normalized to the 28nm process for performance comparison. The conversion methods for process and power are based on references from TSMC ANNUAL REPORT [41].

At 28nm, reaching 1GHz represents a performance inflection point for traditional MAC (TPU-Like). However, due to the constraints of the high bit-width accumulator, it is nearly impossible to maintain a comparable area while under the 0.63 ns clock constraint. As a result, when running at 1.5GHz, traditional MAC experiences a significant increase in area (as illustrated in Figure 9(A)), growing from $367um^2$ to $707um^2$. At this point, the synthesis tool replicates a large amount of logic within the MAC to maintain parallelism and reduce latency. Consequently, for traditional MACs, surpassing 1GHz does not lead to further improvements in area efficiency (as depicted in Figure 9(C)).

In contrast, the latency is independent of bit width in half-adder accumulation schemes (OPT1 $\sim$ OPT4). Therefore, our proposed designs can operate at frequencies above 1.5GHz and achieve high area efficiency. When constrained from 1.0 GHz to 1.5 GHz, the synthetic area of OPT1 is only increased by a factor of 1.14, compared to a factor of 1.93 in TPU-like MACs. This represents a significant improvement in the area efficiency of OPT1 in 1.5GHz.

OPT2 exhibits a similar timing trend but with an increase in area. While OPT2 reduces the area of the reduction logic and output DFFs, it does so at the expense of increased PE bandwidth and it is essential to consider the additional increase in area and power consumption of input DFFs. As a result, OPT2 doesn't offer an advantage over a single PE. However, there are various ways to reduce the average input width of the DFFs in the array level, such as local broadcast and local shared DFFs. Therefore, OPT2 can achieve optimization by sharing input DFFs among multiple PEs through local broadcasting.

OPT3 skips zero PPs. In terms of area analysis for a single PE, similar to OPT2, the inclusion of input DFFs for multiple operands results in a significant occupation of area in a single PE. However, the area and delay of combinatorial logic are significantly reduced. When constrained from 1.5 GHz to 2.0 GHz, the synthetic area of OPT3 increases only by a factor of 1.09 with a peak frequency of 2.5 GHz. From the analysis of area and frequency, it can be concluded that the inflection point of area efficiency performance for OPT3 is above 2.0GHz, and the use of the pre-fetch mechanism in OPT4 effectively addresses the area issue of input DFFs through an external encoder.

In comparison to other bit-slice architectures, OPT3 maintains an area efficiency (in 2GHz) on par with Laconic, 2.12 times that of Bitlet, 5.28 times that of Sibia, and 15.2 times that of Bitwave. Most of these architectures operate at clock frequencies ranging from 250MHz to 1GHz.

It can be observed that bit-serial algorithms typically perform 1-bit or 2-bit parallel multiplications, resulting in extremely low logic area and latency. However, the reduction and accumulation of PPs cause bottlenecks in these architectures, leading to peak frequencies similar to MAC (1GHz). Thus, the key to improving the computational density is to replace the reduction logic with a lighter-weight alternative.

OPT4C and OPT4E are optimizations of OPT3 at the array level. Through sharing 2 encoders among PE columns, making PEs more lightweight, and reducing the input DFFs area through sparse coding prefetching operations. These improvements further enhance the area efficiency compared to OPT3. In addition, OPT4E aims to balance the area ratio between DFFs and joint logic to achieve high area efficiency.

3) Power and energy efficiency: The significant impact of the DFFs and clock network on power analysis can't be overlooked. In high-speed digital circuits, the clock network

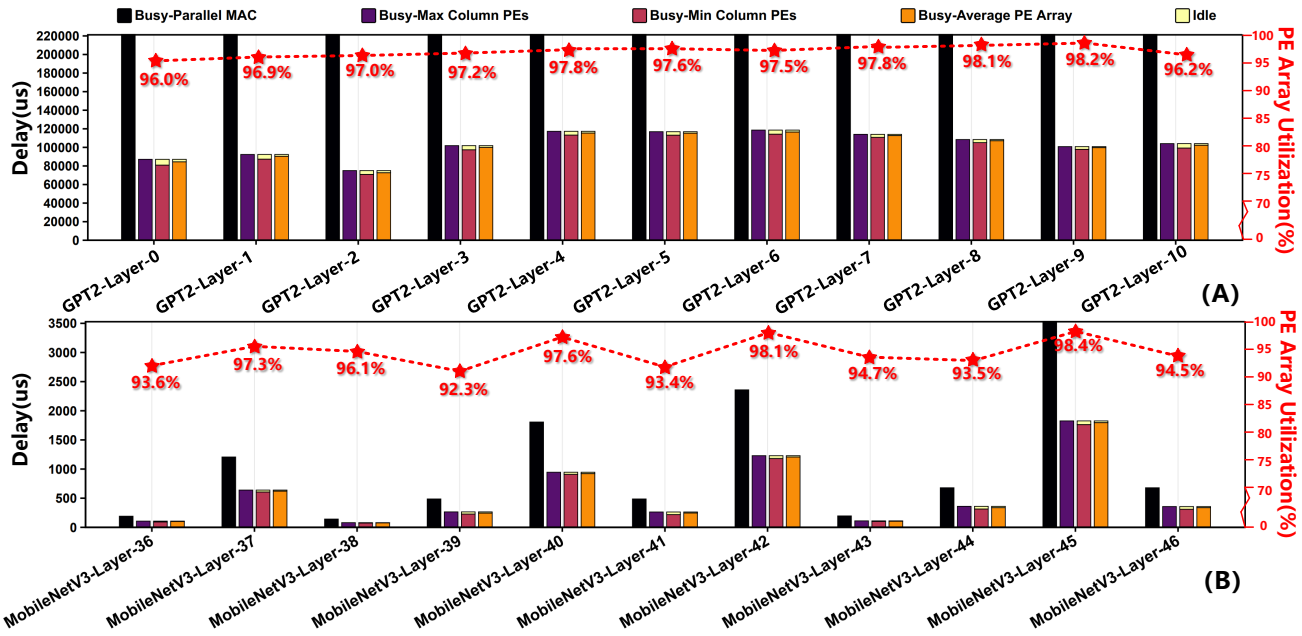

Figure 11: Comparison of the computational performance of a single tile in the TPEs composed of OPT4E and parallel MAC under (A) GPT-2 layer and (B) MobileNetV3 sub-layers, along with an analysis of the utilization of OPT4E array.

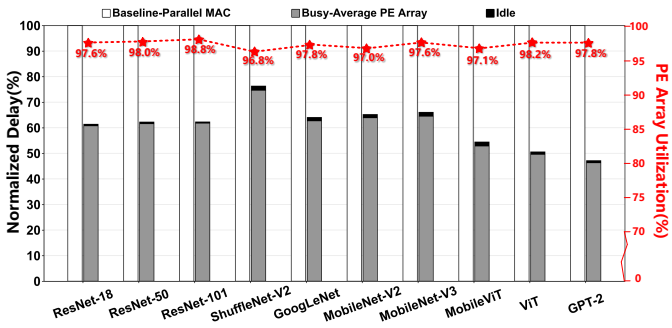

Figure 12: Comparison of performance of TPEs normalization composed of OPT4E and parallel MACs under different networks, and the total idle ratio of OPT4E subarrays.

accounts for 30%∼60% of total power consumption, leaving 40%∼70% to be optimized by logic designers, including DFFs and combinational logic power consumption. Despite optimizations in logic regions such as OPT1, OPT2, and OPT3, the increase in clock network power consumption at high frequencies exceeds the increase in combined logic power consumption. Therefore, when the frequency increases to a certain threshold, the energy efficiency will decrease.

Designers can reduce power consumption at high frequencies by minimizing the register area within the logic design. This consideration is reflected in designs such as OPT4C and OPT4E, which reduces the need for input and output DFFs while balancing the logic and DFFs regions. Ultimately, OPT4E enables significant computational density while maintaining energy efficiency, as demonstrated in Figure 9(B) and (D).

C. Array-level comparison with state of the art

1) Configuration setup: In the experimental deployment of the PE array, since EDA tools require constraint files to be read before synthesis, it is necessary to use predefined delays to constrain the clock. To this end, we thoroughly tested the frequency range for each PE design, as shown in Figure 9, aiming to determine the optimal clock frequency for each configuration (achieving better energy and area efficiency).

From a detailed observation of Figure 9(A), the frequency range of the TPU-like MAC spans from 500 MHz to 1.5 GHz. Beyond 1.5 GHz, timing violations occur, preventing normal operation. Only design 5 (OPT4C) can reach 3.0 GHz, but higher frequencies do not always lead to better synthesis performance.

As shown in Figures 9(C) and 9(D), the TPU-like MAC-based design achieves peak area and energy efficiency at 1.0 GHz. The frequency limit of the PE using the OPT1 design is 2.0 GHz, but its synthesis performance is optimal at 1.5 GHz. Similarly, we identified the optimal frequency points for OPT3, OPT4C, and OPT4E, which were then used as clock constraints for synthesizing and testing the TPEs.

2) Comparision with classical TPE architecture: As depicted in Table VII, we implement the OPT1 on conventional architectures such as TPU (systolic array), Ascend (3D-Cube), Trapezoid (multiplier-adder tree), and FlexFlow (2D-Matrix). For FlexFlow (2D-Matrix), OPT2 is employed. Subsequently, we compare the performance enhancements before and after applying these optimizations, using them as benchmarks. Based on our previous analysis of the area efficiency per PE, we observe an increase in area efficiency across all four

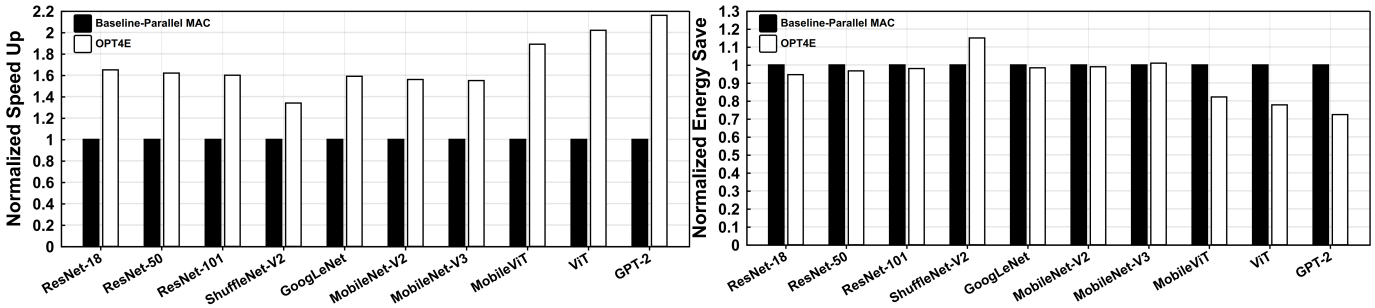

Figure 13: The normalized speedup and energy consumption ratio of TPEs composed of OPT4E and parallel MAC.

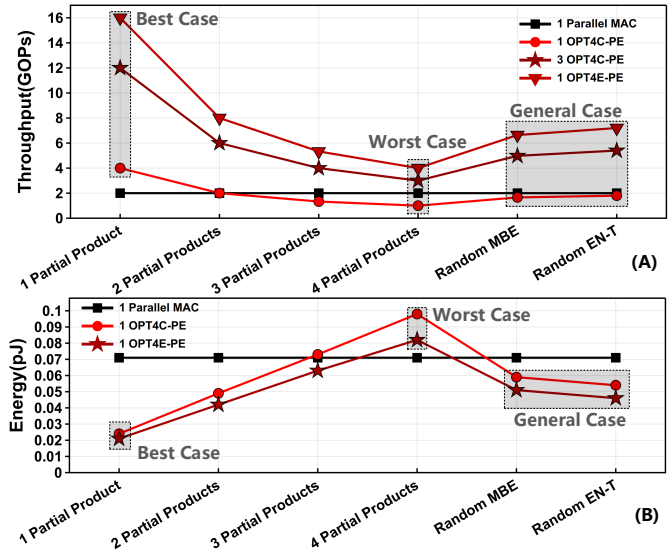

Figure 14: (A) Throughput for different PEs. 1 Parallel MAC ($246um^2$) $\approx$ 3 OPT4C-PE ($81.27um^2$) $\approx$ 1 OPT4E-PE ($311um^2$). (B) Energy consumed per multiplication-accumulation operation.

microarchitectures, by a factor of 1.27, 1.28, 1.58, 1.34 and 1.44, respectively. Energy efficiency was increased by 1.04, 1.56, 1.49, 1.11 and 1.20 times, respectively. Moreover, for the OPT2 particularly in FlexFlow (2D-Matrix), there was a slight improvement over OPT1 which aligns with our previous analysis of PE area efficiency. The reason for this improvement is that the 2D-Matrix architecture broadcasts inputs across its rows and columns, allowing a single input DFFs to be shared among PE rows and columns, leading to a dilution of the area of OPT2's input registers, demonstrating the advantage of having lower bit-widths within PEs.

3) Comparison with the bit-slice architecture: Choosing Laconic as the comparison baseline for bit-slice architecture (Bitlet, Sibia, BitWave, OPT3, OPT4C, and OPT4E in Table VII) reveals a common trait: these methods typically improve energy efficiency significantly but generally lack in area efficiency. Despite their compact size, they aren't as computationally efficient, making it challenging to significantly increase the computational power per unit area. In terms of computational efficiency, the bit-slice technique can be improved in two main ways. First, the number of PPs is reduced by sparse encoding. Second, eliminate the bottleneck of accumulation in bit-slice operations. Hence, the optimization strategy of OPT1 led to the development of OPT3, which effectively addresses these issues and is also applicable to bit-serial processing. Additional advancements include higher-level loop optimizations in arrays such as operand sharing enabling us to propose encoding within all bit-slice PEs to further reduce area and improve timing. Additionally considering the balance between combinational logic and DFFs is a crucial step toward further reducing area efficiency and energy consumption. Finally, our final iteration OPT4E not only maintains commendable energy efficiency but also significantly enhances the computational density of bit-slice architecture.

D. Workloads for DNNs and LLMs

Unlike the TPEs formed by parallel MACs, the throughput of MM provided by the OPT4E is mainly influenced by two factors: (1) The number of partial products after encoding of the multiplicand; (2) the reduction dimension of the vector.

As shown in Figure 14(A), the throughput of parallel MACs is not affected by the number of partial products of the operands. The traditional MACs always parallelly reduce 4 partial products, resulting in constant computational power and energy consumption as shown in Figure 14(B). In contrast, the area of a single PE in the OPT4C ($81.27um^2$) is about one-third of the parallel MAC ($246um^2$). In the best-case scenario, all inputs produce only one partial product after encoding, achieving twice the throughput of a regular MAC with one-third energy savings. In the worst-case scenario, all inputs produce 4 partial products after encoding, resulting in an equivalent computational power of half that of a regular MAC. In more general cases, for a set of normally distributed vectors, the average number of partial products for MBE and EN-T encoding is 2.41 and 2.22 respectively (as shown in Table III). Therefore, a single OPT4C can achieve a throughput close (1.8 GOPS) to that of a regular MAC with lower energy consumption. When comparing equal areas, we used three OPT4Cs and one OPT4E, which generally achieve 2.7$\times$ and 3.6$\times$ the throughput improvement compared to parallel MACs, with lower energy consumption per operation. Even in the worst case, a certain speedup can still be achieved.

The second factor influencing throughput is the dimensionality of the reduction vector. Since synchronization among

different column PEs in OPT4E is necessary after the reduction is completed, a higher vector dimensionality leads to reduced variance in computation time across the column PEs, resulting in improved performance. To illustrate this with practical deep neural networks (DNNs), we select two representative NN layers: the Transformer layer of GPT-2 and the Depthwise (DW)-Pointwise (PW) layer of MobileNet, as depicted in Figures 11 and 12. We employe a systolic array and the OPT4E architecture of the same area for comparing inference delays. We record the fastest computing column PEs (Busy-Min Column PEs), the slowest computing column PEs (Busy-Max Column PEs), and the average busy and idle ratios of all column PEs (Busy-Average PE) for comparison. The specific meaning of delay in Figure 11 refers to the time required for vector reduction under a single excitation (e.g., in a GPT layer, delay represents the inference latency of a single embedding vector at each layer, while for MobileNet, delay refers to the inference time of a single pixel at each layer).

In the multi-head attention layer of GPT-2, which typically involves higher-dimensional matrix multiplication, the idle time has minimal impact on overall computational efficiency. In contrast, MobileNetV3 exhibits a lower reduction dimension in the DW layer and a higher reduction dimension in the PW layer, resulting in lower utilization in the DW layer compared to the PW layer. However, since the computational load of the PW layer is significantly greater than that of the DW layer, a notable speedup can still be achieved across all layers.

As illustrated in Figures 12 and 13, we compared the inference performance of several mainstream backbones. MobileVIT, VIT, and GPT-2 achieved the highest speedup ratios, with performance improvements of 1.89, 2.02, and 2.16 times, respectively. Regarding energy consumption, as shown in Figure 13, networks with higher reduction dimensions tend to achieve greater energy savings.

VI. Disscussion

In actual calculations, column PEs may experience idle times due to early completion of computations. The occurrence of idle periods (bubbles) in column PEs benefits TPEs, as PEs handling vectors with fewer non-zero partial products can quickly complete computations and enter an idle state, saving power. However, processing performance depends on the slowest column PEs. For matrices with higher vector dimensions, the variance in the reduction clock cycles across column PEs gradually decreases. Consequently, as the computation load increases, the bubble ratio also declines. This results in significant benefits from both power consumption and computation speed perspectives. For OPT1 design, all PEs are synchronized throughout the entire computation process. Conversely, for sparse computations encoded in OPT3, OPT4C, and OPT4E, not every MAC clock cycle differs. Since the same multiplicand is broadcast to all column PEs, the reduction cycle for each column PE is identical.

Overall, the MAC plays a critical role in determining the area and performance of AI DSA. Analyzing the MAC components enables the identification of area and delay bottlenecks in each subcomponent, which indirectly impacts TPE performance. In Section III-B, we discuss the validity of component position transformations within nested loops, facilitating the exploration of higher-dimensional transformations in the search space. Furthermore, selecting encoded operands represents an additional dimension in the search space. Prioritizing operands with high sparsity enhances acceleration, further broadening the optimization search space.

VII. Conclusion

Traditional TPE designs primarily focus on data flow reuse through MAC-based specialized matrix multiplication units. This work extends TPE design to the component level within the MAC, identifying bottlenecks through higher-level loop transformations. We first introduce a fine-grained primitive that uncovers a broader design space for TPE. Within this expanded space, we analyze bottlenecks by exposing the implicit dimensions of traditional MAC designs. Subsequently, we apply valid loop transformations across components to address these bottlenecks, resulting in more efficient parallel hardware and providing a methodology for designing high-performance PE microarchitectures. Furthermore, we investigate the principles of bit-sparsity acceleration, where encoded multiplicands enhance operand sparsity, allowing the elimination of zero partial products to achieve sparse acceleration. Leveraging the proposed primitives, we develop a TPE microarchitecture that compresses non-zero partial products, significantly improving performance.

References

[1] "Nvidia tesla v100 gpu architecture white paper," 2017, https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf.

[2] "Chatgpt," 2022, https://openai.com/chatgpt.

[3] "Apple silicon m3 socs," 2023, https://www.anandtech.com/show/21116/apple-announces-m3-soc-family-m3-m3-pro-and-m3-max-make-their-marks.

[4] "Qualcomm brings record-breaking generative ai for devices at snapdragon summit 2023," 2023, https://www.qualcomm.com/news/releases/2023/10/qualcomm-brings-record-breaking-generative-ai-for-devices-at-sna.

[5] J. Albericio, P. Judd, A. Delmás, S. Sharify, and A. Moshovos, "Bit-pragmatic deep neural network computing," 2016. [Online]. Available: https://arxiv.org/abs/1610.06920

[6] O. J. Bedrij, "Carry-select adder," *IRE Transactions on Electronic Computers*, no. 3, pp. 340–346, 1962.

[7] Y. Blumenfeld, I. Hubara, and D. Soudry, "Towards cheaper inference in deep networks with lower bit-width accumulators," *arXiv preprint arXiv:2401.14110*, 2024.

[8] S. Cass, "Taking ai to the edge: Google's tpu now comes in a maker-friendly package," *IEEE Spectrum*, vol. 56, no. 5, pp. 16–17, 2019.

[9] F.-C. Cheng, S. H. Unger, and M. Theobald, "Self-timed carry-lookahead adders," *IEEE Transactions on Computers*, vol. 49, no. 7, pp. 659–672, 2000.

[10] A. Delmas Lascorz, P. Judd, D. M. Stuart, Z. Poulos, M. Mahmoud, S. Sharify, M. Nikolic, K. Siu, and A. Moshovos, "Bit-tactical: A software/hardware approach to exploiting value and bit sparsity in neural networks," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 749–763.

[11] C.-Y. Du, C.-F. Tsai, W.-C. Chen, L.-Y. Lin, N.-S. Chang, C.-P. Lin, C.-S. Chen, and C.-H. Yang, "A 28nm 11.2 tops/w hardware-utilization-aware neural-network accelerator with dynamic dataflow," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 1–3.

[12] A. A. Farooqui and V. G. Oklobdzija, "General data-path organization of a mac unit for vlsi implementation of dsp processors," in *1998 IEEE International Symposium on Circuits and Systems (ISCAS)*, vol. 2. IEEE, 1998, pp. 260–263.

[13] A. Feldmann and D. Sanchez, "Spatula: A hardware accelerator for sparse matrix factorization," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 91–104.

[14] C. Grimm, J. Lee, and N. Verma, "Training neural networks with in-memory-computing hardware and multi-level radix-4 inputs," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.

[15] O. Gustafsson, A. G. Dempster, and L. Wanhammar, "Multiplier blocks using carry-save adders," in *2004 IEEE International Symposium on Circuits and Systems (IEEE Cat. No. 04CH37512)*, vol. 2. IEEE, 2004, pp. II–473.

[16] Z. Huang and M. D. Ercegovac, "High-performance low-power left-to-right array multiplier design," *IEEE Transactions on computers*, vol. 54, no. 3, pp. 272–283, 2005.

[17] D. Im, G. Park, Z. Li, J. Ryu, and H.-J. Yoo, "Sibia: Signed bit-slice architecture for dense dnn acceleration with slice-level sparsity exploitation," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 69–80.

[18] D. Im and H.-J. Yoo, "Lutein: Dense-sparse bit-slice architecture with radix-4 lut-based slice-tensor processing units," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 747–759.

[19] Z. Jia, B. Tillman, M. Maggioni, and D. P. Scarpazza, "Dissecting the graphcore ipu architecture via microbenchmarking," *arXiv preprint arXiv:1912.03413*, 2019.

[20] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles *et al.*, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[21] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.

[22] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[23] S.-R. Kuang, J.-P. Wang, and C.-Y. Guo, "Modified booth multipliers with a regular partial product array," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 56, no. 5, pp. 404–408, 2009.

[24] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 754–768. [Online]. Available: https://doi.org/10.1145/3352460.3358252

[25] G. Li, W. Xu, Z. Song, N. Jing, J. Cheng, and X. Liang, "Ristretto: An atomized processing architecture for sparsity-condensed stream flow in cnn," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1434–1450.

[26] J. Li and Z. Jiang, "Performance analysis of cambricon mlu100," in *Benchmarking, Measuring, and Optimizing: Second BenchCouncil International Symposium, Bench 2019, Denver, CO, USA, November 14–16, 2019, Revised Selected Papers 2*. Springer, 2020, pp. 57–66.

[27] H. Liao, J. Tu, J. Xia, H. Liu, X. Zhou, H. Yuan, and Y. Hu, "Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 789–801.

[28] Y.-C. Lo and R.-S. Liu, "Bucket getter: A bucket-based processing engine for low-bit block floating point (bfp) dnns," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1002–1015. [Online]. Available: https://doi.org/10.1145/3613424.3614249

[29] H. Lu, L. Chang, C. Li, Z. Zhu, S. Lu, Y. Liu, and M. Zhang, "Distilling bit-level sparsity parallelism for general purpose deep learning acceleration," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 963–976.

[30] W. Lu, G. Yan, J. Li, S. Gong, Y. Han, and X. Li, "Flexflow: A flexible dataflow accelerator architecture for convolutional neural networks," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 553–564.

[31] S. Markidis, S. W. Der Chien, E. Laure, I. B. Peng, and J. S. Vetter, "Nvidia tensor core programmability, performance & precision," in *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 2018, pp. 522–531.

[32] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators," *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1160–1174, 2021.

[33] T. Norrie, N. Patil, D. H. Yoon, G. Kurian, S. Li, J. Laudon, C. Young, N. Jouppi, and D. Patterson, "The design process for google's training chips: Tpuv2 and tpuv3," *IEEE Micro*, vol. 41, no. 2, pp. 56–63, 2021.

[34] Y. Pan, J. Yu, A. Lukefahr, R. Das, and S. Mahlke, "Bitset: Bit-serial early termination for computation reduction in convolutional neural networks," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 5s, pp. 1–24, 2023.

[35] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[36] R. Prabhakar, S. Jairath, and J. L. Shin, "Sambanova sn10 rdu: A 7nm dataflow architecture to accelerate software 2.0," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 350–352.

[37] M. R. Santoro and M. A. Horowitz, "Spim: a pipelined 64* 64-bit iterative multiplier," *IEEE journal of solid-state circuits*, vol. 24, no. 2, pp. 487–493, 1989.

[38] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 304–317.

[39] M. Shi, V. Jain, A. Joseph, M. Meijer, and M. Verhelst, "Bitwave: Exploiting column-based bit-level sparsity for deep learning acceleration," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 732–746.

[40] M. Sjalander and P. Larsson-Edefors, "High-speed and low-power multipliers using the baugh-wooley algorithm and hpm reduction tree," in *2008 15th IEEE international conference on electronics, circuits and systems*. IEEE, 2008, pp. 33–36.

[41] Taiwan Semiconductor Manufacturing Company, "Tsmc 2023 annual report," 2023, accessed: 2024-06-27. [Online]. Available: https://investor.tsmc.com/static/annualReports/2023/english/pdf/2023_tsmc_ar_e_ch7.pdf

[42] S. Veeramachaneni, K. M. Krishna, L. Avinash, S. R. Puppala, and M. Srinivas, "Novel architectures for high-speed and low-power 3-2, 4-2 and 5-2 compressors," in *20th International Conference on VLSI Design held jointly with 6th International Conference on Embedded Systems (VLSID'07)*. IEEE, 2007, pp. 324–329.

[43] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Transactions on electronic Computers*, no. 1, pp. 14–17, 1964.

[44] G. Wang, S. Cai, W. Li, D. Lyu, and G. He, "Bsvit: A bit-serial vision transformer accelerator exploiting dynamic patch and weight bit-group quantization," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.

[45] Q. Wu, Y. Gui, Z. Zeng, X. Wang, H. Liang, and X. Jin, "En-t: Optimizing tensor computing engines performance via encoder-based methodology," in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, 2024, pp. 608–615.

[46] J. Yang, Z. Zhang, Z. Liu, J. Zhou, L. Liu, S. Wei, and S. Yin, "Fusekna: Fused kernel convolution based accelerator for deep neural networks," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 894–907.

[47] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina, C. Kozyrakis, and M. Horowitz, "Interstellar: Using halide's scheduling language to analyze dnn accelerators," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 369–383. [Online]. Available: https://doi.org/10.1145/3373376.3378514

[48] Y. Yang, J. Emer, and D. Sanchez, "Trapezoid: A versatile accelerator for dense and sparse matrix multiplications," in *in Proceedings of the 51th annual International Symposium on Computer Architecture (ISCA-51)*, 2024.

[49] W.-C. Yeh and C.-W. Jen, "High-speed booth encoded parallel multiplier design," *IEEE Transactions on Computers*, vol. 49, no. 7, pp. 692–701, 2000.

[50] S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang, "Tileflow: A framework for modeling fusion dataflow via tree-based analysis," in *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 1271–1288.